



DATABASE SYSTEM

PROJECT REPORT

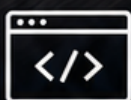
A **MySQL** Database Project for a
Formula-Style Racing Championship



**5 CONNECTED
TABLES**



**75 SQL
SCENARIOS**



ADVANCED SQL

- JOINS
- CTEs
- WINDOW FUNCTIONS
- SUBQUERIES
- VIEWS



**REAL-WORLD
RACING DATA**



**USER PRIVILEGES
& SECURITY**



PROJECT

ApexGP Racing Database System



DBMS

MySQL 8.0



AUTHOR

Satyajit Pradhan



DATE

May 2025



TOOLS USED

MySQL Workbench, DBML, DrawDB



PROJECT TYPE

Academic / Portfolio Project

APEX GRAND PRIX RACING LEAGUE

Championship Timing Database — Project Report

Relational Database Design, Query Engineering & Data Governance

1. Executive Summary

The Apex Grand Prix (AGP) is a global open-wheel racing championship in which ten teams field a combined forty drivers across a multi-circuit season. Every lap of every session — practice, qualifying, and the race itself — is timed to the thousandth of a second, and that data drives real-time decisions for race engineers, team strategists, the broadcast crew, and league administrators.

This project delivers the relational database that underpins that workflow. The database holds roughly 30,000 records across five interconnected tables, the large majority of which are individual lap times. This report documents the schema design, summarizes the 75 operational requirements gathered from stakeholders, and groups them into the SQL and database-engineering skill areas needed to satisfy each one — from everyday reporting to performance tuning, data integrity, and access control.

2. Database Schema & Design

The database is organized around five tables that mirror how a racing championship actually operates: teams field drivers, drivers race at circuits, circuits host races, and races generate lap times. The entity-relationship diagram below shows the primary keys (yellow), foreign keys, and the cardinality of each relationship.

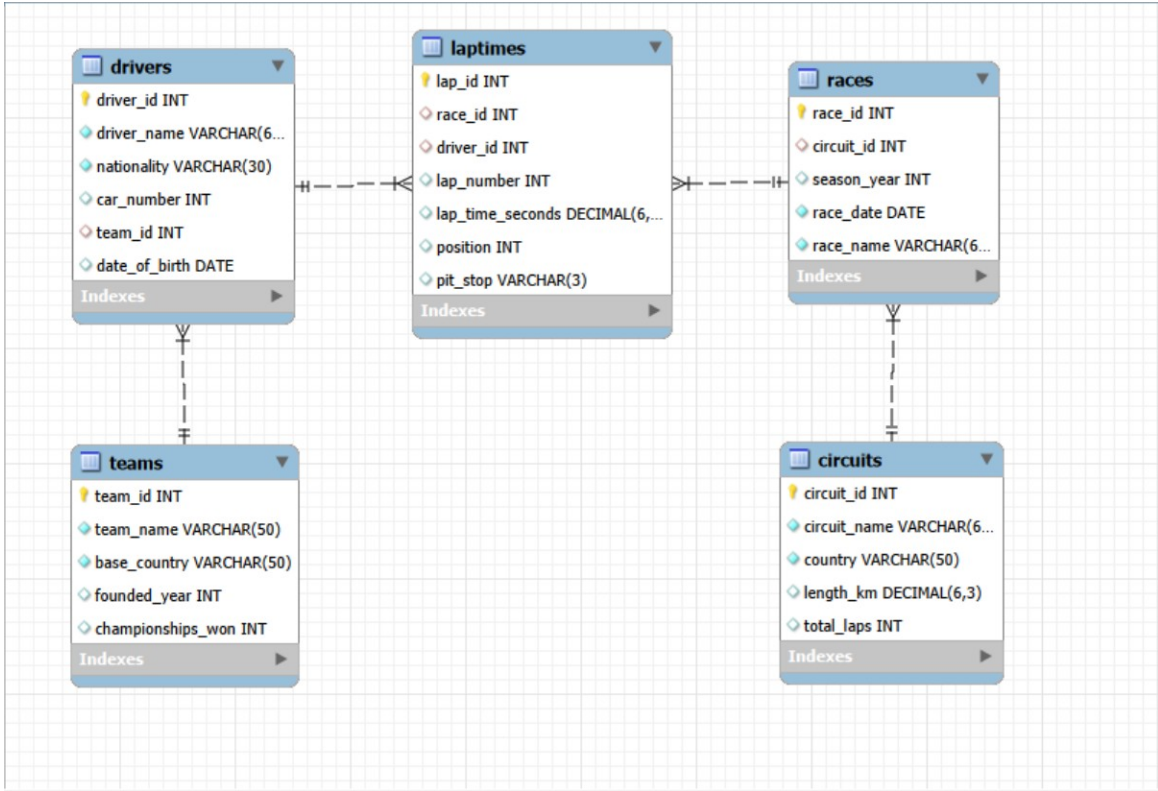


Figure 1. Entity-relationship diagram for the AGP database.

2.1 Table Overview

Table	Purpose	Key Relationships
teams	One row per constructor (10 teams)	1 team → many drivers
drivers	One row per driver (up to 4 per team)	1 driver → many lap times
circuits	One row per track on the calendar	1 circuit → many races
races	One row per race held at a circuit	1 race → many lap times
laptimes	One row per recorded lap (largest table)	Links drivers ↔ races

2.2 Relationship Notes

- teams (1) — (many) drivers: each driver belongs to exactly one team; a team may field several drivers.
- drivers (1) — (many) lap-times and races (1) — (many) lap-times: every lap is tied to one driver and one race, so lap-times is the fact table at the centre of the schema.
- circuits (1) — (many) races: a circuit can host races across multiple seasons; every race must reference a valid circuit.

3. Methodology

Stakeholders across engineering, strategy, broadcast, and commercial teams contributed 75 real-world situations that the database must support. Each case was analysed against the schema to identify what it was actually asking for — a query, a schema change, an integrity rule, a performance fix, or an access control decision — and grouped by the underlying database concept it exercises. The table below summarizes these groupings; several cases legitimately touch more than one concept and are listed under their primary skill area.

3.1 Case Classification by Database Concept

SQL / Database Concept	# Cases	Representative Case Numbers
Aggregation, GROUP BY & HAVING	17	4, 6, 8, 9, 14, 18, 19, 31, 34, 37, 43, 47, 50, 56, 59, 61, 70
Window functions (LAG/LEAD, RANK, NTILE, running & moving averages)	9	1, 10, 16, 30, 52, 60, 63, 65, 66
Joins & handling missing data (LEFT/FULL joins, orphan checks)	9	3, 7, 11, 20, 39, 49, 55, 57, 67
Subqueries & CTEs (multi-stage logic)	7	9, 28, 35, 40, 45, 53, 58
Conditional logic / CASE labelling	4	15, 29, 38, 46
Date & time functions	5	21, 33, 48, 69, 71
String functions & data cleaning	3	41, 54, 73
Constraints (NOT NULL, FK, CHECK, UNIQUE)	5	25, 26, 62, 64, 75
Indexing & performance tuning	3	12, 32, 68
Views for recurring reporting needs	3	2, 44, 72
Data manipulation (UPDATE/DELETE at scale)	4	13, 17, 22, 27
Access control & security (GRANT/REVOKE, roles)	3	24, 36, 74
Schema evolution (ALTER TABLE)	1	5

4. Key Findings

- Reporting and aggregation dominate the requirement set (roughly a third of all cases), reflecting how heavily the strategy and broadcast teams rely on season, driver, and circuit-level summaries.
- A significant share of cases (window functions, running totals, moving averages) require lap-by-lap and race-by-race sequencing rather than simple aggregates — a direct consequence of lap-times being an ordered, high-volume fact table.
- Data quality and integrity concerns (orphaned lap records, inconsistent capitalization, stray whitespace, invalid finishing positions or negative lap times) recur throughout, underscoring the need for constraints enforced at the schema level rather than caught after the fact.
- Performance considerations — indexing driver_id lookups and the (race_id, lap_number) composite filter — become material once the lap-times table is understood to hold the majority of the database's 30,000 records.
- Operational governance needs (contract terminations, read-only analyst accounts, an elevated timing-officer role) show that access control is treated as a first-class requirement alongside the query workload.

5. Conclusion & Next Steps

The five-table schema is compact but sufficient to answer the full range of stakeholder needs: it supports everyday reporting through views, historical and live analysis through aggregation and window functions, data trustworthiness through constraints, responsiveness at scale through targeted indexing, and controlled access through role-based permissions.

The recommended next step is to implement the categorized solutions above as a reviewed SQL script — starting with the constraints and indexes that protect data integrity and query performance, followed by the views and reporting queries that serve day-to-day stakeholder requests.